
Nhắc lại kiến thức bài trước

Xây dựng lớp



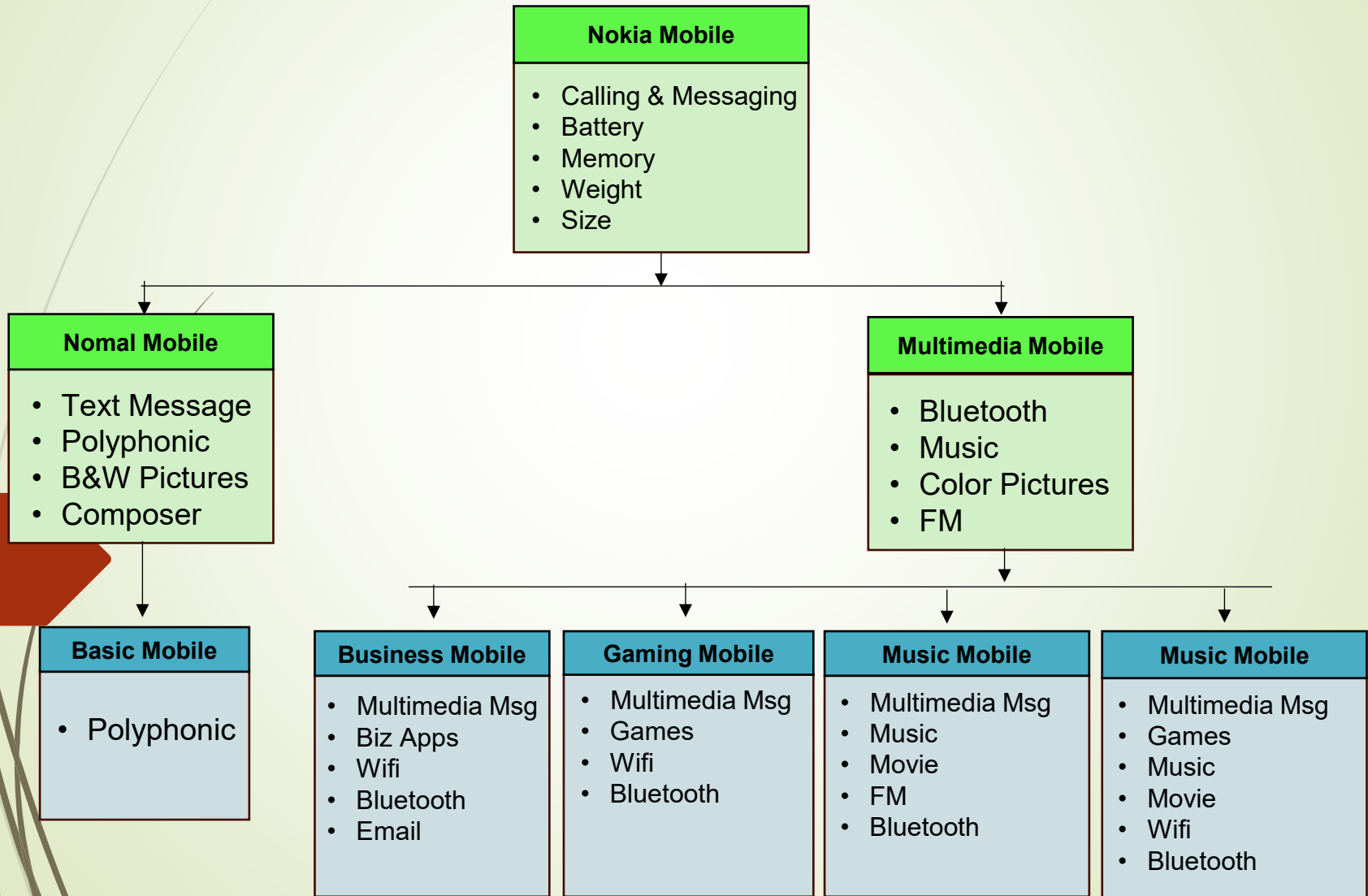
Trừu tượng hóa dữ liệu



Trừu tượng hóa dữ liệu

Just for
reference

- Trừu tượng hóa các đối tượng điện thoại Nokia

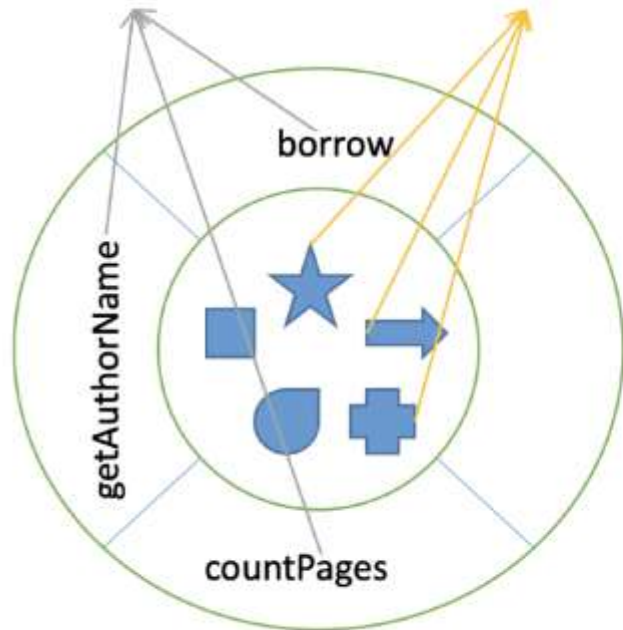


Các thành phần của lớp

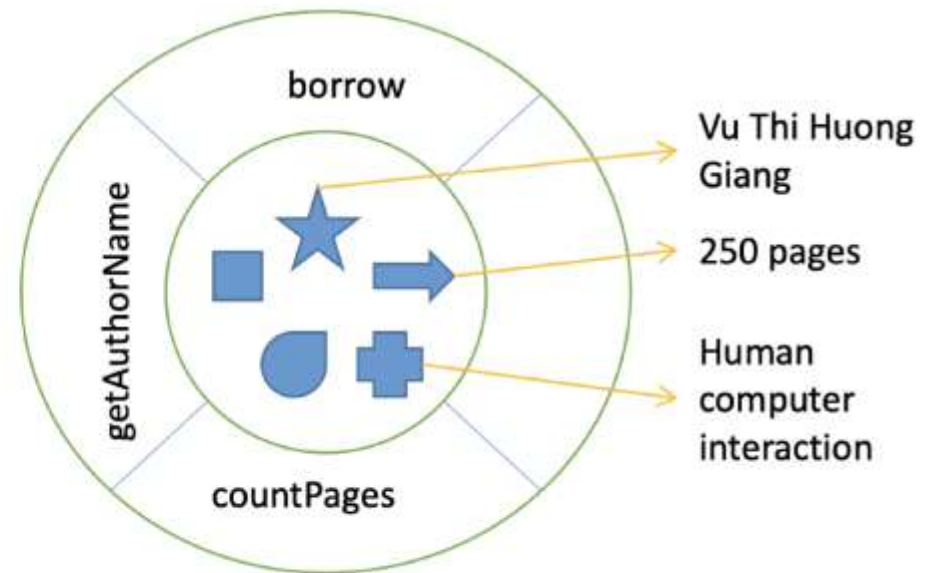
- Lớp đóng gói các phương thức và thuộc tính chung của các đối tượng cùng một loại

Phương thức: các hành vi đối tượng có thể thực hiện

Thuộc tính: các thông tin liên quan đến trạng thái



Thể hiện: Một đối tượng cụ thể
Thuộc tính thể hiện: gán giá trị cho các thuộc tính của một đối tượng cụ thể



Ví dụ: Thuộc tính & Phương thức

Thực lễ

Khai báo lớp

Khai báo thuộc tính

Tự tham chiếu

Khai báo phương thức

Khởi tạo đối tượng

Gán giá trị cho thuộc tính

Gọi phương thức của đối tượng

```
class Point:
    x = int
    y = int

    def print_point(self):
        print ('(%d, %d)' % (self.x, self.y))

p1 = Point()
p1.x = 3
p1.y = 5
p1.print_point()

-----
Kết quả: (3, 5)
```

Chỉ định truy cập

- Chỉ định truy cập:
 - public**: Lớp có thể được truy cập từ bất cứ đâu, kể cả bên ngoài package chứa lớp đó.
 - private**: Lớp chỉ có thể được truy cập trong phạm vi lớp đó.
 - mặc định: Lớp có thể được truy cập từ bên trong package chứa lớp đó.

inner class

	public	mặc định	private
Cùng lớp	✓	✓	✓
Cùng gói	✓	✓	✗
Khác gói	✓	✗	✗

```
class Point:  
    x = int  
    y = int
```

```
class Point:  
    x = int  
    y = int
```

access modifier

Chỉ định truy cập

- Trong biểu đồ lớp, dấu **-** được sử dụng để thể hiện chỉ định truy cập private
- Dấu **+** được sử dụng để thể hiện chỉ định truy cập public

Professor

- name
- employeeID : UniqueId
- hireDate
- status
- discipline
- maxLoad

+ submitFinalGrade()
+ acceptCourseOffering()
+ setMaxLoad()
+ takeSabbatical()
+ teachClass()

Đối tượng

Đối tượng là gì?

"Tất cả mọi thứ đều là đối tượng"

-Alan Kay



Chữa bài tập

Xây dựng lớp



OOP04: Các kỹ thuật xây dựng lớp



Nội dung

1. Đóng gói
2. Hàm khởi tạo
3. Nạp chồng
4. Kết tập



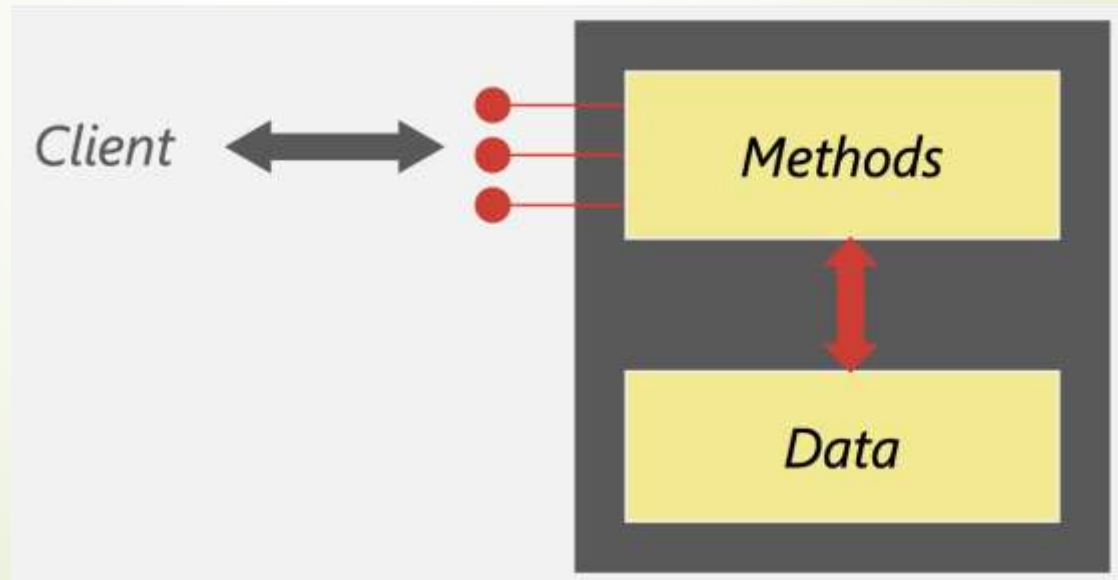
1

Đóng gói

Encapsulation

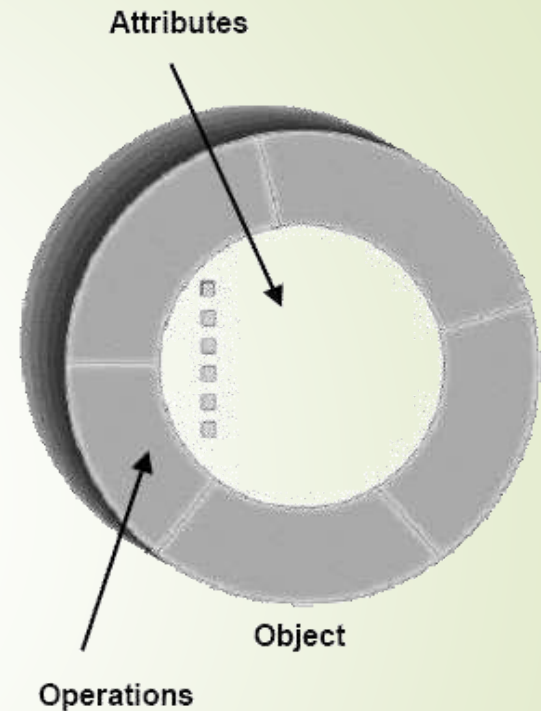
Đóng gói

- Một đối tượng có hai khung nhìn:
 - Bên trong: Chi tiết về các thuộc tính và các phương thức của lớp tương ứng với đối tượng
 - Bên ngoài: Các dịch vụ mà một đối tượng có thể cung cấp và cách đối tượng đó tương tác với phần còn lại của hệ thống



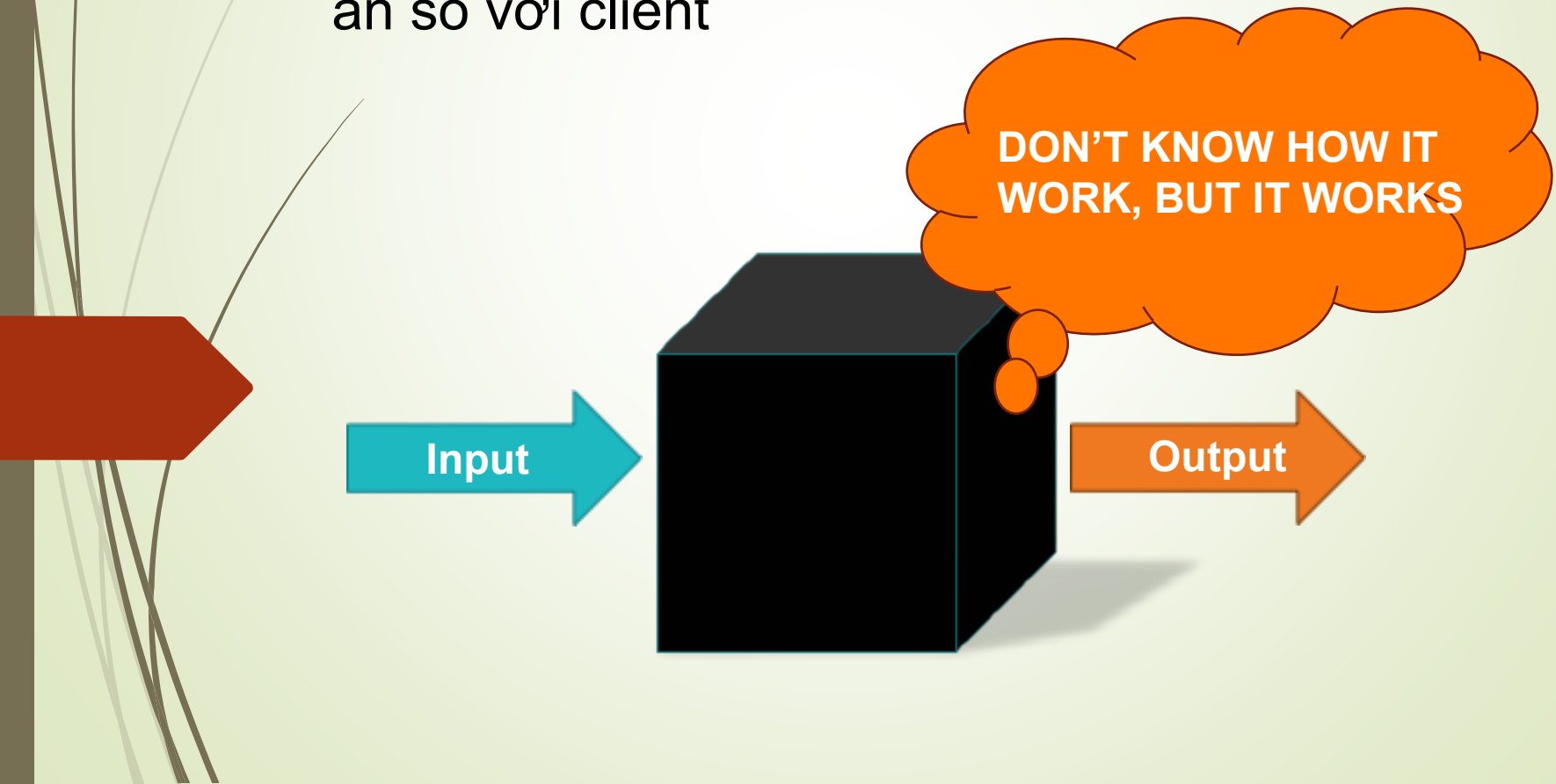
Đóng gói

- Đóng gói (Encapsulation)
 - Dữ liệu và phương thức được đóng gói trong một lớp
 - Dữ liệu được che giấu ở bên trong lớp và chỉ được truy cập và thay đổi ở các phương thức public



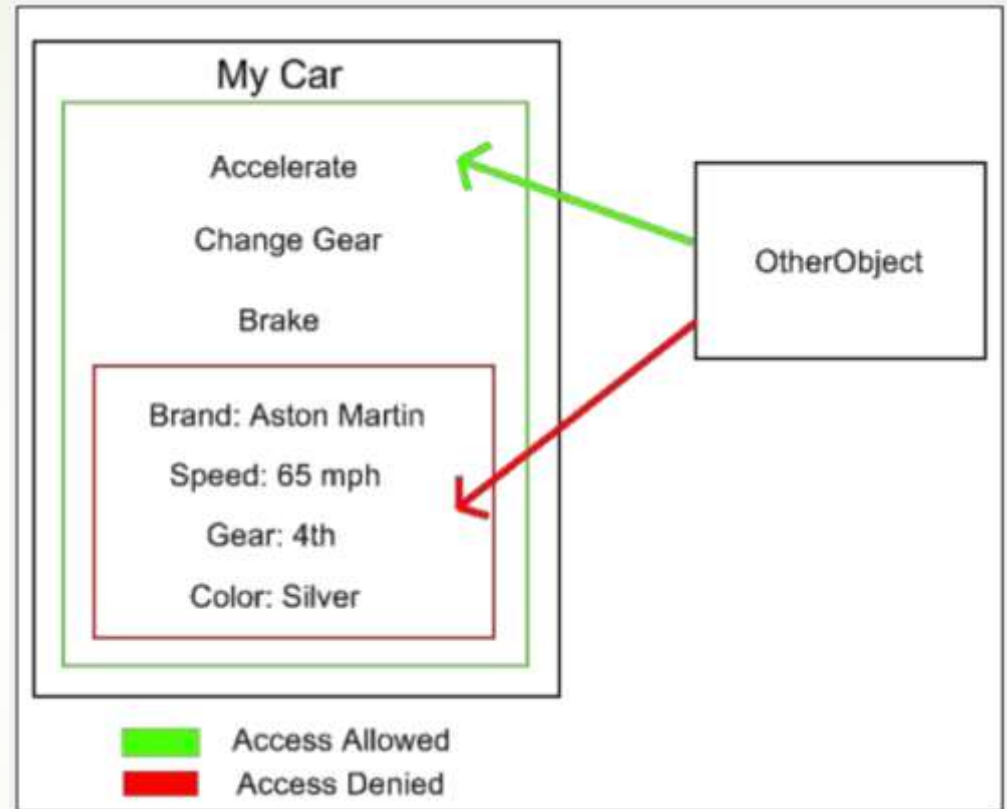
Đóng gói

- Một đối tượng là một thực thể được đóng gói, cung cấp tập các dịch vụ nhất định
- Một đối tượng được đóng gói có thể được xem như một hộp đen – các công việc bên trong là ẩn so với client



Che giấu dữ liệu

- Dữ liệu được che giấu ở bên trong lớp và chỉ được truy cập và thay đổi thông qua các phương thức public của lớp
 - Tránh thay đổi trái phép hoặc làm sai lệch dữ liệu

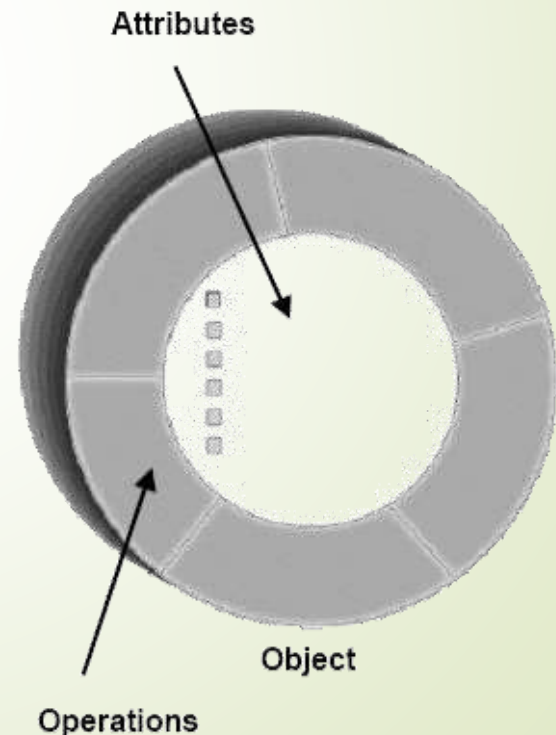


Che giấu dữ liệu

- Các thành viên dữ liệu
 - Chỉ có thể truy cập từ các phương thức bên trong lớp
 - Chỉ định truy cập là private để bảo vệ dữ liệu
- Các đối tượng khác muốn truy nhập vào dữ liệu riêng tư này phải thông qua các phương thức public

BankAccount

- owner: String
- balance: Float
+ debit(Float): boolean
+ credit(Float)



Cơ chế che giấu dữ liệu

- Các thành phần dữ liệu là private -> Để truy cập và chỉnh sửa các giá trị của dữ liệu, lớp cần phải cung cấp các dịch vụ
 - Accessor (getter): Trả về giá trị hiện tại của một thuộc tính (dữ liệu)
 - Mutator (setter): Thay đổi giá trị của một thuộc tính
 - Thường là getX và setX, trong đó x là tên thuộc tính



```
class BankAccount:  
    __owner = str  
    __balance = 0.0
```

```
def getOwner(self):  
    return self.__owner
```

Phương thức Get

- Các phương thức truy vấn **Get** là các phương thức dùng để hỏi giá trị của các thành viên dữ liệu của một đối tượng
- Có nhiều loại câu hỏi truy vấn có thể:
 - truy vấn đơn giản (“giá trị của x là bao nhiêu?”)
 - truy vấn điều kiện (“thành viên x có lớn hơn 10 không?”)
 - truy vấn dẫn xuất (“tổng giá trị của các thành viên x và y là bao nhiêu?”)
- Đặc điểm quan trọng của phương thức truy vấn là nó không nên thay đổi trạng thái hiện tại của đối tượng
 - không thay đổi giá trị của thành viên dữ liệu nào.

Phương thức Set

- Các phương thức thiết lập **Set** là các phương thức dùng để thay đổi giá trị các thành viên dữ liệu
- Ưu điểm của việc sử dụng các phương thức setter là kiểm soát tính hợp lệ của các thành phần dữ liệu
 - Kiểm tra giá trị đầu vào trước khi gán vào các thuộc tính

Ví dụ

```
class Time:
    def __init__(self):
        self.__hours = 0
        self.__minutes = 0
        self.__seconds = 0

    def set_Hours(self, hours):
        self.__hours = hours

    def set_Minutes(self, minutes):
        self.__minutes = minutes

    def set_Seconds(self, seconds):
        self.__seconds = seconds

    def get_Hours(self):
        return self.__hours

    def get_Minutes(self):
        return self.__minutes

    def get_Seconds(self):
        return self.__seconds

    def print_time(self):
        print(f"{self.__hours}:{self.__minutes}:{self.__seconds}")
```

restricted access: private
members are *not*
externally accessible; but
we need to know and
modify their value

set method: public
methods that allow
clients to *modify private*
data; also known as
mutators

get method: public
methods that allow
clients to *read private*
data; also known as
accessor

```
# Creating an instance of Time class
time = Time()
# Using set methods to set the values of private attributes
time.set_Hours(10)
time.set_Minutes(30)
time.set_Seconds(50)
# Using get methods to retrieve the values of private attributes
hours = time.get_Hours()
minutes = time.get_Minutes()
seconds = time.get_Seconds()
# Printing the values of private attributes using print_time method
time.print_time()
```


Bài tập

- Viết mã nguồn cho lớp NhanVien như trong hình bên biết:

- Lương = Lương cơ bản * Hệ số lương
- Phương thức inTTin() hiển thị thông tin của đối tượng NhanVien tương ứng.

```
NhanVien
-tenNhanVien: String
-luongCoBan: double
-heSoLuong: double
+LUONG_MAX: double
+tangLuong(double): boolean
+ tinhLuong(): double
+inTTin()
```

- Phương thức tangLuong tăng hệ số lương hiện tại lên một lượng bằng giá trị tham số truyền vào. Nếu điều này làm cho lương của nhân viên > lương tối đa cho phép thì không cho phép thay đổi, in ra thông báo và trả về false, ngược lại trả về true.
- Viết các phương thức **get** và **set** cho các thuộc tính của lớp NhanVien

2

Hàm khởi tạo

Constructor

Khởi tạo đối tượng

- Để khởi tạo đối tượng: Trong python rất đơn giản
`acc = BankAccount()`

Phương thức khởi tạo ngầm định

- Phương thức khởi tạo **ngầm định (implicit)** sẽ tự động gán các giá trị mặc định cho các thành phần dữ liệu
 - Được thực hiện trước khi lập trình viên có thể tác động lên đối tượng
 - Nếu không muốn sử dụng phương thức khởi tạo ngầm định -> có thể viết phương thức khởi tạo cụ thể (explicit) cho lớp

Phương thức khởi tạo

- Là phương thức được gọi để gán các giá trị cho các thành phần dữ liệu khi đối tượng được khởi tạo
 - Tên của phương thức trùng tên lớp
 - KHÔNG có kiểu dữ liệu trả về
 - Có thể có hoặc không có tham số
- Phương thức khởi tạo KHÔNG được coi là thành viên của lớp
- Python sẽ không sử dụng phương thức khởi tạo ngầm định khi lập trình viên đã viết (ghi đè) phương thức khởi tạo cho lớp

Phương thức khởi tạo

- Ví dụ phương thức khởi tạo không có tham số (còn gọi là phương thức khởi tạo **mặc định**)

```
class BankAccount:  
    __owner = str  
    __balance = float  
  
    def __init__(self):  
        self.__owner = "  
        self.__balance = 0.0
```

Phương thức khởi tạo

- Ví dụ phương thức khởi tạo có tham số

```
class BankAccount:  
    __owner = str  
    __balance = float  
  
    def __init__(self, name, balance):  
        self.__owner = name  
        self.__balance = balance
```

- Mục đích: giúp khởi tạo đối tượng dễ dàng hơn

```
account = BankAccount("Kiên", 2000000 )
```

3

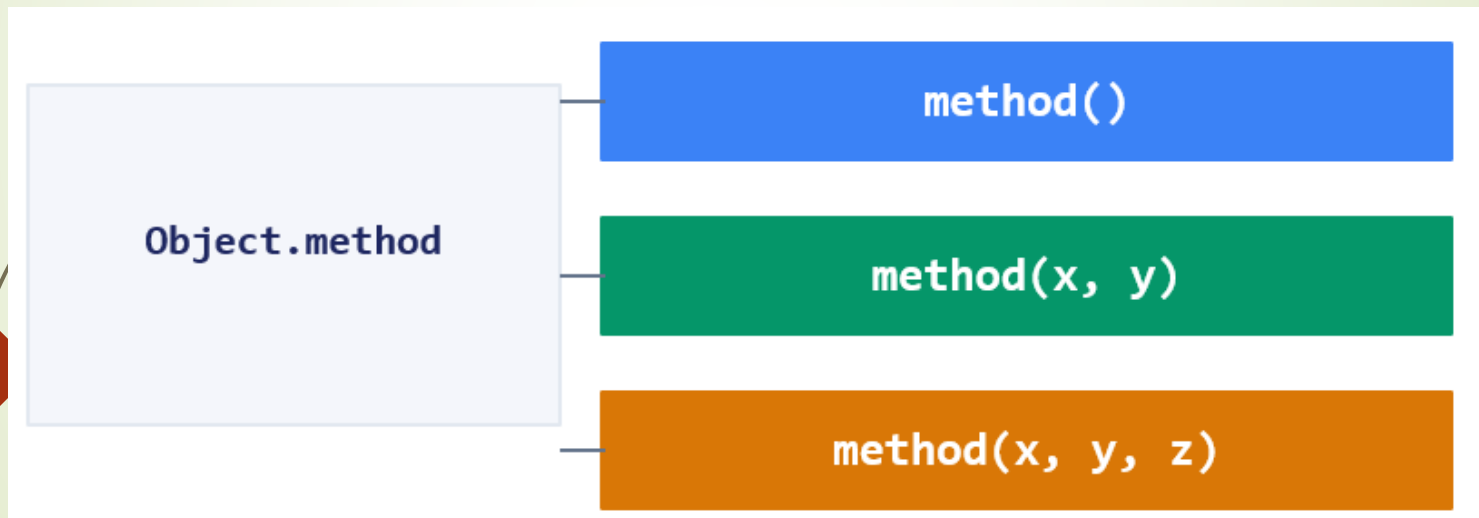


Nạp chồng

Overloading

Nạp chồng phương thức là gì?

- Hai hoặc nhiều phương thức có cùng tên nhưng khác số lượng tham số hoặc khác loại tham số hoặc cả hai. Các phương thức này được gọi là nạp chồng phương thức (**overloaded methods**) và điều này được gọi là nạp chồng (**overloading**).



Lưu ý:

Python KHÔNG hỗ trợ nạp chồng theo cách Java/C++. Khai báo hai hàm cùng tên → hàm sau ghi đè hàm trước, không báo lỗi.

Java / C++ vs Python - Triết lý khác nhau

Tiêu chí	Java / C++	Python
Cơ chế	Compile-time Kiểm tra khi biên dịch, dựa trên signature	Runtime Một hàm duy nhất xử lý nhiều trường hợp bên trong
Khai báo 2 hàm cùng tên	Hợp lệ — trình biên dịch phân biệt theo tham số	Hàm sau ghi đè hàm trước Không báo lỗi!
Cách khuyến nghị	Khai báo tường minh nhiều hàm	Dùng *args, tham số mặc định, hoặc kiểm tra kiểu bên trong hàm
Hỗ trợ ngôn ngữ	Tích hợp sẵn — native support	multipledispatch (thư viện bên thứ ba, không phải chuẩn)

Duck typing trong Python: "nếu nó có thể bơi và kêu quạc quạc, thì nó là vịt"
— kiểu dữ liệu xác định qua hành vi, không phải khai báo tường minh.

Vấn đề: Hàm sau ghi đè hàm trước

Code có vẻ đúng...

```
def product(a, b):  
    p = a * b  
    print(p)
```

```
def product(a, b, c):  
    p = a * b * c  
    print(p)
```

```
product(4, 5)      # → TypeError!  
product(4, 5, 5)   # → 100
```

Python xử lý như thế nào:

1

Định nghĩa product(a, b)

Python lưu tên product vào namespace, trở tới hàm 2-tham-số.

2

Định nghĩa product(a, b, c)

Python ghi đè tên product → hàm 2-tham-số biến mất hoàn toàn.

3

Gọi product(4, 5)

Chỉ còn hàm 3-tham-số. Python không tìm được → TypeError!

Signature (Chữ ký) của phương thức

Khái niệm nền

Signature = tên phương thức + số lượng tham số + kiểu dữ liệu của từng tham số (theo thứ tự)

```
product(int a, int b)
```

Kiểu trả về (void, int, float...) KHÔNG thuộc signature

Tên hàm

Kiểu tham số 1

Kiểu tham số 2

Ba signature khác nhau của cùng tên hàm product:

✓ `product(int a, int b)` Tên: product | 2 tham số | kiểu: (int, int)

✓ `product(int a, int b, int c)` Tên: product | 3 tham số | kiểu: (int, int, int)

✓ `product(float a, float b)` Tên: product | 2 tham số | kiểu: (float, float)

Java/C++: trình biên dịch đọc signature → chọn đúng hàm khi gọi. Python: chỉ 1 hàm tồn tại — signature không được dùng để phân biệt.

Đọc Signature bằng Python — inspect module

Python cung cấp module **inspect** để in signature của hàm tại runtime — không cần đoán, không cần nhớ.

• code.py

```
import inspect

# Định nghĩa 3 hàm để thử nghiệm
def product(a, b):
    return a * b

def product_3(a, b, c):
    return a * b * c

def product_f(a: float, b: float) -> float:
    return a * b

# In signature của từng hàm
funcs = [product, product_3, product_f]
for f in funcs:
    sig = inspect.signature(f)
    print(f.__name__, "|", sig)

# Chi tiết từng tham số
for name, p in sig.parameters.items():
    print(name, "|", p.annotation)
```

► Output

```
product    | (a, b)
product_3  | (a, b, c)
product_f  | (a: float,
                b: float) -> float

# params product_f:
a | <class 'float'>
b | <class 'float'>
```

①

inspect.signature(fn)

Trả về Signature chứa đủ: tên tham số, kiểu dữ liệu, giá trị mặc định, kiểu trả về.

②

sig.parameters

OrderedDict tên → Parameter. Dùng .items() để lặp từng tham số.

③

param.annotation

Type hint khai báo. Không có → trả về inspect.Parameter.empty.



Ứng dụng:

Dùng inspect để viết decorator tự validate tham số, sinh documentation tự động, hoặc debug khi không nhớ thứ tự tham số của hàm.

Ví dụ

Hãy cài đặt lớp **LineSegment** biểu diễn các Đoạn thẳng bao gồm các thuộc tính và phương thức sau:

- **Thuộc tính:** kiểu truy cập private
 - d1, d2: là hai điểm đầu mút của đoạn thẳng, là đối tượng thuộc lớp Point
- **Phương thức:** kiểu truy cập public
 - Hàm xây dựng mặc định, không đối số
 - Tạo đoạn thẳng với đầu mút d1 có tọa độ (8, 5) và d2 có tọa độ (1, 0)
 - Hàm xây dựng có đối số: **LineSegment(Point d1, Point d2)**
 - Tạo đoạn thẳng với đầu mút d1 và d2
 - Lấy d1 và d2 làm hai điểm đầu mút, không tạo thêm điểm mới.
 - Hàm xây dựng 4 đối số: **LineSegment(int x1, int y1, int x2, int y2)**
 - Tạo đoạn thẳng có đầu mút d1(x1, y1) và d2(x2, y2)
 - Hàm xây dựng sao chép: **LineSegment(LineSegment S)**
 - Sao chép sâu đối tượng S.

Phương án 1 — Dùng *args

Pythonic nhất

Một hàm duy nhất, xử lý mọi số lượng tham số. Đây là cách Python khuyến nghị.

```
class LineSegment:
    def __init__(self, *args):

        # Không đối số: dùng điểm mặc định
        if len(args) == 0:
            self.__d1 = Point(8, 5)
            self.__d2 = Point(1, 0)

        # 2 đối tượng Point
        elif len(args) == 2:
            if isinstance(args[0], Point):
                self.__d1 = args[0]
                self.__d2 = args[1]

        # 4 tọa độ (x1,y1,x2,y2)
        elif len(args) == 4:
            self.__d1 = Point(args[0], args[1])
            self.__d2 = Point(args[2], args[3])
```

Cách dùng

Không đối số

```
s1 = LineSegment()
```

→ $d1=(8,5)$, $d2=(1,0)$

2 điểm Point

```
s2 = LineSegment(p1, p2)
```

→ 2 điểm cho sẵn

4 tọa độ

```
s3 = LineSegment(0,0,3,4)
```

→ Tạo từ tọa độ

Một hàm duy nhất. Không ghi đè. Pythonic và dễ bảo trì.

Phương án 2 — Tham số mặc định

Đơn giản

Dùng None làm giá trị mặc định, kiểm tra bên trong hàm. Phù hợp khi số lượng trường hợp ít.

```
def add(a=None, b=None):  
    if a is not None and b is None:  
        print(a)      # Chỉ 1 tham số  
    else:  
        print(a + b)  # 2 tham số  
  
add(2, 3)  # → 5  
add(2)     # → 2
```

Ưu điểm

Đơn giản, không cần thư viện ngoài. Đọc và viết nhanh với 2–3 trường hợp.

Khi nào nên dùng?

- Hàm khởi tạo với ít biến thể
- Hàm tiện ích nhỏ, tối đa 2–3 trường hợp
- Khi muốn code dễ đọc, không cần thư viện

Nhược điểm

Khó mở rộng khi nhiều trường hợp. Logic điều kiện trong hàm phình to, khó bảo trì.

Phương án 3 — multipledispatch

Thư viện bên thứ ba

Cú pháp gần với Java nhất — dùng decorator @dispatch để phân biệt theo kiểu dữ liệu.

```
pip install multipledispatch
```

```
from multipledispatch import dispatch
```

```
@dispatch(int, int)
def product(a, b):
    print(a * b)
```

```
@dispatch(int, int, int)
def product(a, b, c):
    print(a * b * c)
```

```
@dispatch(float, float)
def product(a, b):
    print(a * b)
```

```
product(2, 3)           # 6
product(2, 3, 2)        # 12
product(1.5, 2.0)       # 3.0
```

Phù hợp khi

Cần phân biệt theo kiểu dữ liệu tường minh (int vs float vs str).
Logic phức tạp.

Nhược điểm

Thư viện bên thứ ba — cần pip install. Không phải chuẩn Python, hiệu năng có thể bị ảnh hưởng.

Lưu ý:

multipledispatch dùng để minh họa khái niệm. Trong thực tế nên ưu tiên *args (Phương án 1) hoặc tham số mặc định (Phương án 2).

LineSegment: Java vs Python — cùng bài toán, khác cú pháp

● Java — 4 constructor riêng biệt

```
class LineSegment {
    private Point d1, d2;

    // Constructor 1: mặc định
    public LineSegment() {
        d1 = new Point(8, 5);
        d2 = new Point(1, 0);
    }

    // Constructor 2: 2 điểm Point
    public LineSegment(Point p1, Point p2) {
        d1 = p1; d2 = p2;
    }

    // Constructor 3: 4 tọa độ
    public LineSegment(int x1,y1, int x2,y2) {
        d1 = new Point(x1,y1);
        d2 = new Point(x2,y2);
    }

    // Constructor 4: sao chép
    public LineSegment(LineSegment s) {
        d1=s.d1; d2=s.d2;
    }
}
```

4 constructor riêng biệt · trình biên dịch chọn theo signature

vs

● Python — 1 hàm duy nhất với *args

```
class LineSegment:
    __d1 = __d2 = None

    def __init__(self, *args):

        # Constructor 1: mặc định
        if len(args)==0:
            self.__d1=Point(8,5)
            self.__d2=Point(1,0)

        # Constructor 2: 2 điểm Point
        elif len(args)==2:
            if isinstance(args[0],Point):
                self.__d1=args[0]
                self.__d2=args[1]

        # Constructor 3: 4 tọa độ
        elif len(args)==4:
            self.__d1=Point(args[0],args[1])
            self.__d2=Point(args[2],args[3])

        # Constructor 4: sao chép
        elif len(args)==1:
            if isinstance(args[0],LineSegment):
                self.__d1=args[0].__d1
                self.__d2=args[0].__d2
```

1 hàm, if/elif bên trong · runtime kiểm tra len() và isinstance()

copy.deepcopy — sao chép sâu trong Python

⚠ Shallow copy — dùng chung bộ nhớ

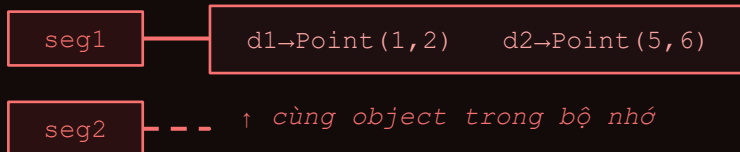
```
import copy

p1 = Point(1, 2)
p2 = Point(5, 6)
seg1 = LineSegment(p1, p2)

# Shallow: chỉ copy tham chiếu
seg2 = seg1

# Thay đổi seg1 → ảnh hưởng seg2!
seg1.__d1.x = 99
print(seg2.__d1.x) # → 99 ⚠
```

Memory Model



⚠ sửa seg1 → seg2 bị ảnh hưởng

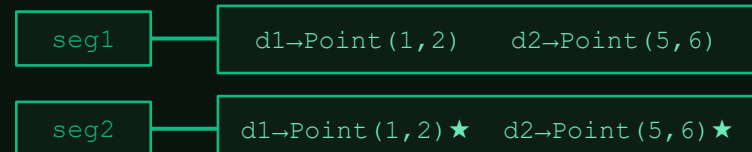
✓ Deep copy — hoàn toàn độc lập

```
import copy

p1 = Point(1, 2)
p2 = Point(5, 6)
seg1 = LineSegment(p1, p2)

# Deep: tạo bản sao hoàn toàn mới
seg2 = copy.deepcopy(seg1)

# Thay đổi seg1 → KHÔNG ảnh hưởng seg2
seg1.__d1.x = 99
print(seg2.__d1.x) # → 1 ✓
```



★ bản sao độc lập — sửa seg1 không ảnh hưởng

Cách gọi constructor — Java vs Python

Cùng 4 kịch bản khởi tạo — Java dùng 4 cú pháp riêng, Python dùng 1 hàm duy nhất:

Java	VS	Python
1. Constructor mặc định <code>LineSegment s1 = new LineSegment();</code> <code>// d1=(8,5), d2=(1,0)</code>		<code>s1 = LineSegment()</code> <code># d1=(8,5), d2=(1,0)</code>
2. Hai điểm Point cho sẵn <code>Point p1=new Point(0,0), p2=new Point(3,4);</code> <code>LineSegment s2 = new LineSegment(p1, p2);</code>		<code>p1, p2 = Point(0,0), Point(3,4)</code> <code>s2 = LineSegment(p1, p2)</code>
3. Bốn tọa độ (x1,y1,x2,y2) <code>LineSegment s3 = new LineSegment(0,0,3,4);</code> <code>// trình dịch chọn đúng constructor 3</code>		<code>s3 = LineSegment(0,0,3,4)</code> <code># len(args)==4 → vào nhánh elif thứ 3</code>
4. Sao chép sâu (copy constructor) <code>LineSegment s4 = new LineSegment(s3);</code> <code>// trình dịch chọn đúng constructor 4</code>		<code>s4 = LineSegment(s3)</code> <code># len(args)==1 & isinstance → copy</code>
Trình biên dịch tự chọn đúng constructor dựa vào số lượng và kiểu tham số truyền vào (signature matching).		Cú pháp gọi giống hệt nhau — Python phân biệt kịch bản bằng len(args) và isinstance() bên trong hàm.

Tổng kết — Khi nào dùng phương án nào?

Phương án 1

Khuyến nghị

`*args`

Số lượng tham số biến thiên. Cần xử lý nhiều trường hợp đầu vào khác nhau.

Phương án 2

Đơn giản

Default params

Tình huống đơn giản, tối đa 2–3 trường hợp. Đọc và viết nhanh.

Phương án 3

Nâng cao

`multipledispatch`

Cần phân biệt theo kiểu dữ liệu tường minh. Khi môi trường cho phép thư viện ngoài.

Phương án	Cú pháp chính	Pythonic?	Phụ thuộc thư viện
<code>*args</code>	Kiểm tra <code>len(args)</code> hoặc <code>isinstance()</code>	Có	Không
Default params	Tham số <code>=None</code> , kiểm tra trong hàm	Có	Không
<code>multipledispatch</code>	Decorator <code>@dispatch(type, type)</code>	Tương đối	<code>pip install</code>

4

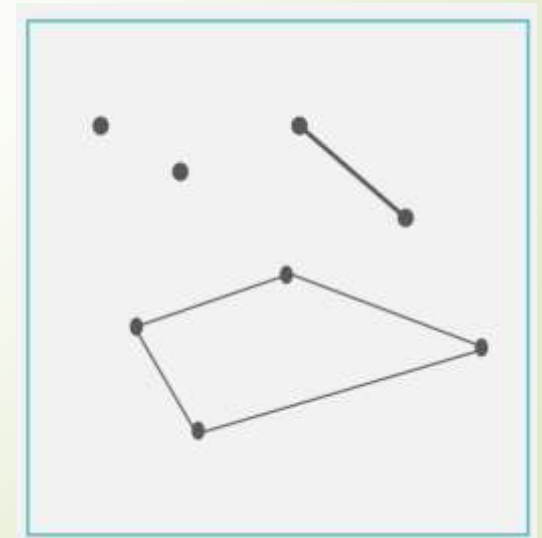


Kết tập

Aggregation

Mối quan hệ kết tập

- Lớp toàn thể chứa các đối tượng của lớp thành phần
 - Đối tượng lớp thành phần: Là một phần (is-a-part of) của lớp toàn thể
 - Quan hệ chứa/có ("has-a") hoặc là một phần ("is-a-part- of")
- Ví dụ
 - Tứ giác gồm 4 Điểm
 - Ô tô gồm 4 Bánh xe
 - Lớp học chứa 91 Sinh viên



Ví dụ

```
class Point:
    x = int
    y = int
    # Khai báo hàm khởi tạo
    def __init__(self, x=0, y=0):
        self.x = x
        self.y = y
    def __str__(self):
        return("(%d, %d)" % (self.x, self.y))

class Rect:
    __d1 = Point
    __d2 = Point
    __d3 = Point
    __d4 = Point
    def __init__(self, p1=Point(0,0), p2=Point(0,1), p3=Point(1,1), p4=Point(1,0)):
        self.__d1 = p1
        self.__d2 = p2
        self.__d3 = p3
        self.__d4 = p4
    def __str__(self):
        return "[%s, %s, %s, %s]" % (self.__d1, self.__d2, self.__d3, self.__d4)

# Kiểm tra kết quả:
rc1 = Rect()
rc2 = Rect(Point(2,2), Point(2,3), Point(3,3), Point(3,2))

print(rc1)
print(rc2)
-----
Kết quả:
-----
[(0, 0), (0, 1), (1, 1), (1, 0)]
[(2, 2), (2, 3), (3, 3), (3, 2)]
```

Bản chất của kết tập

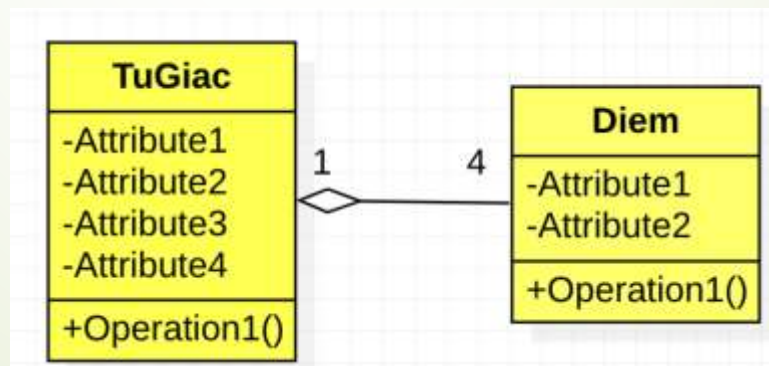
- Kết tập (aggregate)
 - Các thành phần của lớp mới là các đối tượng của các lớp có sẵn.
 - Tái sử dụng các thành phần dữ liệu và các hành vi của lớp thành phần thông qua **đối tượng** của lớp thành phần
- Lớp mới
 - Lớp toàn thể (Aggregate/Whole)
- Lớp cũ
 - Lớp thành phần (Part).

Thứ tự khởi tạo

- Khi một đối tượng được tạo mới, các thuộc tính của đối tượng đó đều phải được khởi tạo và gán những giá trị tương ứng.
- Các đối tượng thành phần được khởi tạo trước
 - ➔ Các phương thức khởi tạo của các lớp của các đối tượng thành phần được thực hiện trước

Biểu diễn kết tập bằng UML

- Sử dụng “hình thoi” tại đầu của lớp toàn thể
- Sử dụng bội số quan hệ (multiplicity) tại 2 đầu, có thể sử dụng:
 - Một số nguyên dương: 1, 2,...
 - Dải số: (0..1, 2..4)
 - *: Bất kỳ số nào
 - Không có: Mặc định là 1



Tổng kết

OOP04: Các kỹ thuật xây dựng lớp

1. Đóng gói (Encapsulation)

- Dữ liệu + phương thức được gói trong một lớp
- Che giấu dữ liệu (data hiding): thuộc tính private, truy cập qua getter/setter
- Getter không thay đổi trạng thái đối tượng; Setter kiểm tra tính hợp lệ trước khi gán

2. Hàm khởi tạo (Constructor)

- `__init__` được gọi tự động khi tạo đối tượng
- Không có kiểu trả về, không phải thành viên của lớp
- Hai dạng: không tham số (mặc định) và có tham số.

3. Nạp chồng (Overloading)

- Python KHÔNG hỗ trợ nạp chồng như Java/C++ — hàm sau ghi đè hàm trước
- Khái niệm Signature: tên hàm + số lượng + kiểu tham số phương án thay thế trong Python: `*args` (khuyến nghị), tham số mặc định, `multipledispatch`
- Ví dụ LineSegment: 1 hàm `__init__` xử lý 4 kịch bản qua `len(args)` + `isinstance()`

4. Kết tập (Aggregation)

- Quan hệ "has-a" / "is-a-part-of": lớp toàn thể chứa đối tượng của lớp thành phần
- Ví dụ: Tứ giác gồm 4 Point, LineSegment gồm 2 Point
- Đối tượng thành phần được khởi tạo trước đối tượng toàn thể
- UML: hình thoi + multiplicity

Lưu ý: Shallow copy vs Deep copy (`copy.deepcopy`) — khi nào cần tạo bản sao độc lập để tránh chia sẻ tham chiếu.

Bài tập

- Viết mã nguồn cho lớp NhanVien như trong hình bên biết:

- Lương = Lương cơ bản * Hệ số lương
- Phương thức inTTin() hiển thị thông tin của đối tượng NhanVien tương ứng.

```
NhanVien  
-tenNhanVien: String  
-luongCoBan: double  
-heSoLuong: double  
+LUONG_MAX: double  
+tangLuong(double): boolean  
+tinhLuong(): double  
+inTTin()
```

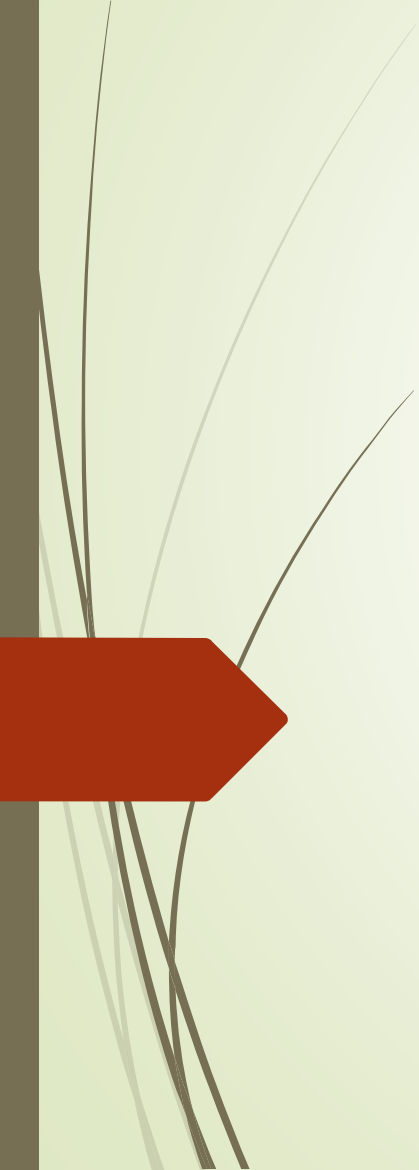
- Phương thức tangLuong tăng hệ số lương hiện tại lên một lượng bằng giá trị tham số truyền vào. Nếu điều này làm cho lương của nhân viên > lương tối đa cho phép thì không cho phép thay đổi, in ra thông báo và trả về false, ngược lại trả về true.

- Viết các phương thức **get** và **set** cho các thuộc tính của lớp NhanVien

Bài tập

Hãy cài đặt lớp **LineSegment** biểu diễn các Đoạn thẳng bao gồm các thuộc tính và phương thức sau:

- **Thuộc tính:** kiểu truy cập private
 - d1, d2: là hai điểm đầu mút của đoạn thẳng, là đối tượng thuộc lớp Point
- **Phương thức:** kiểu truy cập public
 - Hàm xây dựng mặc định, không đối số
 - Tạo đoạn thẳng với đầu mút d1 có tọa độ (8, 5) và d2 có tọa độ (1, 0)
 - Hàm xây dựng có đối số: **LineSegment(Point d1, Point d2)**
 - Tạo đoạn thẳng với đầu mút d1 và d2
 - Lấy d1 và d2 làm hai điểm đầu mút, không tạo thêm điểm mới.
 - Hàm xây dựng 4 đối số: **LineSegment(int x1, int y1, int x2, int y2)**
 - Tạo đoạn thẳng có đầu mút d1(x1, y1) và d2(x2, y2)
 - Hàm xây dựng sao chép: **LineSegment(LineSegment S)**
 - Sao chép sâu đối tượng S.

A decorative graphic on the left side of the slide featuring several thin, curved lines representing grass or reeds. At the bottom left, there is a solid red arrow pointing to the right.

Thank you!

Any questions?